MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

aculty Name	Nitin Gandhi		
ubject Name with Code	Database Management Systems(4CS4-04)		
emester	4th	Session	2025-26 (Even)
ranch	CSE	Batch	B
nit No.	3	Submission Date	23/1/2026

ecture No.	Topic Covered		Total Pages
1	Introductions to Schema Refinement	1-3	3
2	Functional Dependencies	4-7	4
3	Normal Forms and First ,Second form of Normalization	8-11	4
4	Boyce-Codd Normal Forms, Third Normal Form	12-16	5
5	Forth Normal Form	17-18	2
6	Normalization-Decomposition into BCNF Decomposition into 3-NF	19-22	4

References:	
1	H. F. Korth and Silberschatz: Database Systems Concepts, McGraw Hill
2	Almasri and S. B. Navathe: Fundamentals of DataBase Systems
3	Ramakrishnan: Database Management Systems
4	C. J. Date: Data Base Design, Addison Wesley
5	Hansen and Henson: DBM and Design, PHI

Faculty

HOD

Dean Academics

15/1/26

UNIT-3 (Schema Refinement & Normal Form)

Introductions to Schema Refinement

LECTURE NO: -1

Schema Refinement

Database schemas tell the database what your data is and how to work with it. Database schemas help the database engine understand these patterns which allows it to enforce constraints on the data, respond with the right information when queried, and manipulate it in ways that users request.
Good schemas tend to reduce implicit information in favour of making it visible to the system
and its users. Schemas in relational databases can reduce information redundancy, ensure data
consistency, and provide the scaffolding and structures needed to access and join related data.

Defining physical vs logical schemas

When talking about designing database schemas, a logical schema is a general design for organizing data into different categories, defining properties of the data, and determining the best structure for database items.
A physical schema is recognized as being the next step in the design process where implementation-specific details are worked out.
Schema refinement is just a fancy term for saying polishing tables. It is the last step before considering physical design with typical workloads:

1) Requirement analysis: user needs
2) Conceptual design: high-level description, often using E/R diagrams
3) Logical design: from graphs to tables (relational schema)
4) Schema refinement: checking tables for redundancies and anomalies

Let's see an example of redundancies and anomalies. Consider the following table where the client's name is the primary key.

employeeID	FirstName	LastName	Hire Date	Client
1	Mary	Marley	1/1/2005	James H.
2	Robert	Crosby	2/4/2008	Clark L.
3	Robert	Crosby	2/4/2008	Tina M.

The table is presenting information on employees (sales reps) and their clients.

If we want to **insert data**, we notice that:

1.each row requires an entry in the client field.

2.we can't insert data for newly hired sales reps until they've been assigned to one or more clients.

3.if sales reps are in a training process, even if they've been already hired, they can't actually join the database because they need to have a delegated client… unless "dummy" clients are created.

If we want to **update data**, we notice that:

1.the sales reps name is repeated for each client.

2.what if, for a given client, we misspelled the name of the sales reps Crosby instead of Cosby… how can we edit that without affecting all the sales reps called Crosby?

If we want to **delete data**, what if Mary doesn't have a client anymore because she's taking a year off? We are forced to either

1.create a dummy client

2.incorrectly showing her with a client she no longer handled

3.delete Mary's record (even if however, she's still an employee)

4. notice we cannot have "null" as a client since primary field keys cannot store null.

When we have to treat with schema refinement, we often notice that the main problem is redundancy. In order to identify schemas with such problems, we'll introduce the notion of functional dependencies: a relationship that exists when one attribute uniquely determines another attribute. A functional dependency is simply a new type of constraint between two attributes.

Say that R is a relation with attributes X and Y, we say that there is a functional dependency X ->Y when Y is functionally dependent on X (where X is the determinant set and Y is the dependent attribute.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-3 (Schema Refinement & Normal Form)

Functional Dependency

LECTURE NO: -2

What is Functional Dependency?

Functional Dependency (FD) is a constraint that determines the relation of one attribute to another attribute in a Database Management System (DBMS). Functional Dependency helps to maintain the quality of data in the database. It plays a vital role to find the difference between good and bad database design.

A functional dependency is denoted by an arrow "→". The functional dependency of X on Y is represented by X → Y. Let's understand Functional Dependency in DBMS with example.

Example:

Employee number	Employee Name	Salary	City
1	Dana	50000	San Francisco
2	Francis	38000	London
3	Andrew	25000	Tokyo

In this example, if we know the value of Employee number, we can obtain Employee Name, city, salary, etc. By this, we can say that the city, Employee Name, and salary are functionally depended on Employee number.

Key Terms Description

Axiom Axioms is a set of inference rules used to infer all the functional dependencies on a relational database.

Decomposition It is a rule that suggests if you have a table that appears to contain two entities which are determined by the same primary key then you should consider breaking them up into two different tables.

Dependent It is displayed on the right side of the functional dependency diagram.

Determinant It is displayed on the left side of the functional dependency Diagram.

Union It suggests that if two tables are separate, and the PK is the same, you

should consider putting them. together

Rules of Functional Dependencies

Below are the Three most important rules for Functional Dependency in Database:

Reflexive rule : If X is a set of attributes and Y is_subset_of X, then X holds a value of Y.

Augmentation rule: When x -> y holds, and c is attribute set, then ac -> bc also holds. That is adding attributes which do not change the basic dependencies.

Transitivity rule: This rule is very much similar to the transitive rule in algebra if x -> y holds and y -> z holds, then x -> z also holds. X -> y is called as functionally that determines y.

Types of Functional Dependencies in DBMS

There are mainly four types of Functional Dependency in DBMS. Following are the types of Functional Dependencies in DBMS:

1. Trivial functional dependency
2. Non-Trivial functional dependency
3. Multivalued functional dependency
4. Transitive functional dependency

1. Trivial Functional Dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant.
i.e. If X → Y and Y is the subset of X, then it is called trivial functional dependency.
For example,

rollno	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, {roll_no, name} → name is a trivial functional dependency, since the dependent name is a
subset of determinant set {roll_no, name}
Similarly, roll_no → roll_no is also an example of trivial functional dependency.

2. Non-trivial Functional Dependency

In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant.

i.e. If X → Y and Y is not a subset of X, then it is called Non-trivial functional dependency.

For example,

rollno	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, roll_no → name is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no. Similarly, {roll_no, name} → age is also a non-trivial functional
dependency, since age is not a subset of {roll_no, name}

3. Multivalued Functional Dependency

Multivalued dependency occurs in the situation where there are multiple independent multivalued attributes in a single table. A multivalued dependency is a complete constraint between two sets of attributes in a relation. It requires that certain tuples be present in a relation.

In Multivalued functional dependency, entities of the dependent set are not dependent on each other.

i.e. If a → {b, c} and there exists no functional dependency between b and c, then it is called a
multivalued functional dependency.

For example,

rollno	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, roll_no → {name, age} is a multivalued functional dependency,

4. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant.

i.e. If a → b & b → c, then according to axiom of transitivity, a → c. This is a transitive functional dependency.
For example,

Enrno	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Here, enrol_no → dept and dept → building_no,

Hence, according to the axiom of transitivity, enrol_no → building_no is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

References: -
1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-3 (Schema Refinement & Normal Form)

Normalization(1NF & 2NF)

LECTURE NO: -3

What is Normalization?

Normalization is a database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies. Normalization rules divides larger tables into smaller tables and links them using relationships. The purpose of Normalisation in SQL is to eliminate redundant (repetitive) data and ensure data is stored logically.

Normalization is used for mainly two purposes,

1.Eliminating redundant(useless) data.

2.Ensuring data dependencies make sense i.e data is logically stored.

Problems Without Normalization

If a table is not properly normalized and have data redundancy then it will not only eat up extra memory space but will also make it difficult to handle and update the database, without facing data loss. Insertion, Updation and Deletion Anomalies are very frequent if database is not normalized. To understand these anomalies let us take an example of a Student table.

rollno	name	branch	hod	office_tel
401	Akon	CSE	Mr. X	53337
402	Bkon	CSE	Mr. X	53337
403	Ckon	CSE	Mr. X	53337
404	Dkon	CSE	Mr. X	53337

In the table above, we have data of 4 Computer Sci. students. As we can see, data for the fields branch, hod(Head of Department) and office_tel is repeated for the students who are in the same branch in the college, this is Data Redundancy.

There are three types of anomalies that occur when the database is not normalized. These are – Insertion, update and deletion anomaly. Let's take an example to understand this.

Example: Suppose a manufacturing company stores the employee details in a table named employee that has four attributes: emp_id for storing employee's id, emp_name for storing employee's name, emp_address for storing employee's address and emp_dept for storing the department details in which the employee works. At some point of time the table looks like this:

emp_id	emp_name	emp_address	emp_dept
101	Rick	Delhi	D001
101	Rick	Delhi	D002
123	Maggie	Agra	D890
166	Glenn	Chennai	D900
166	Glenn	Chennai	D004

The above table is not normalized. We will see the problems that we face when a table is not normalized.

Update anomaly: In the above table we have two rows for employee Rick as he belongs to two departments of the company. If we want to update the address of Rick then we have to update the same in two rows or the data will become inconsistent. If somehow, the correct address gets updated in one department but not in other then as per the database, Rick would be having two different addresses, which is not correct and would lead to inconsistent data.

Insert anomaly: Suppose a new employee joins the company, who is under training and currently not assigned to any department then we would not be able to insert the data into the table if emp_dept field doesn't allow nulls.

Delete anomaly: Suppose, if at a point of time the company closes the department D890 then deleting the rows that are having emp_dept as D890 would also delete the information of employee Maggie since she is assigned only to this department.

Normal Form Description

1NF A relation is in 1NF if it contains an atomic value.

2NF A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.

3NF A relation will be in 3NF if it is in 2NF and no transition dependency exists.

4NF A relation will be in 4NF if it is in Boyce Codd normal form and has no multi valued dependency.

5NF A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.

For a table to be in the First Normal Form, it should follow the following 4 rules:
1. It should only have single(atomic) valued attributes/columns.
2. Values stored in a column should be of the same domain
3. All the columns in a table should have unique names.

emp_id	emp_name	emp_address	emp_mobile
101	Herschel	New Delhi	8912312390
102	Jon	Kanpur	8812121212,9900012222
103	Ron	Chennai	7778881212
104	Lester	Bangalore	9990000123,8123450987

Two employees (Jon & Lester) are having two mobile numbers so the company stored them in the same field as you can see in the table above.

This table is not in 1NF as the rule says “each attribute of a table must have atomic (single) values”, the emp_mobile values for employees Jon & Lester violates that rule.
To make the table complies with 1NF we should have the data like this:

emp_id	emp_name	emp_address	emp_mobile
101	Herschel	New Delhi	8912312390
102	Jon	Kanpur	8812121212
102	Jon	Kanpur	9900012222
103	Ron	Chennai	7778881212
104	Lester	Bangalore	9990000123
104	Lester	Bangalore	8123450987

Second normal form (2NF)

A table is said to be in 2NF if both the following conditions hold:
1. Table is in 1NF (First normal form)
2.In the second normal form, all non-key attributes are fully functional dependent on the primary key.

Key Attribute: -
Uniquely identify a record or an instance on an entity. Required to retrieve a particular record from the entity table. non-prime (non-key) attributes are the attributes other than

the primary key attributes. They can store a value any many times.

STUDENT (RegNo, Name, Address)

RegNo is primary key (Key or prime Attribute)

Name and address are Non-Key attribute

Let's assume, a school can store the data of teachers and the subjects they teach. In a school,

a teacher can teach more than one subject.

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Computer	38

In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-3 (Schema Refinement & Normal Form)

Normalization(BCNF & 3NF)

LECTURE NO: -4

A transitive dependency in a database is an indirect relationship between values in the same table that causes a functional dependency. To achieve the normalization standard of Third Normal Form (3NF), you must eliminate any transitive dependency.
{Book} ->{Author} (if we know the book, we know the author name)
{Author} -> {Author_age} Therefore, as per the rule of transitive dependency:
{Book} -> {Author_age} should hold,

Book	Author	Author_age
Game of Thrones	George R. R. Martin	66
Harry Potter	J. K. Rowling	49
Dying of the Light	George R. R. Martin	66

Third Normal Form
A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.
A relation is in third normal form if it holds at least one of the following conditions for every non-trivial function dependency X → Y.
X is a super key.
Y is a prime attribute, i.e., each element of Y is part of some candidate key.
Super Key: The set of attributes which can uniquely identify a tuple is known as Super Key.
For Example, STUD_NO, (STUD_NO, STUD_NAME) etc.
• Adding zero or more attributes to candidate key generates super key.
• A candidate key is a super key but vice versa is not true.

Candidate Key: The minimal set of attributes which can uniquely identify a tuple is known as candidate key.

For Example, STUD_NO in STUDENT relation.

• The value of Candidate Key is unique and non-null for every tuple.

• There can be more than one candidate key in a relation.

Example of Super & Candidate Key

STUDENT

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY	STUD_AGE
1	RAM	9716271721	Haryana	India	20
2	RAM	9898291281	Punjab	India	19
3	SUJIT	7898291981	Rajsthan	India	18
4	SURESH		Punjab	India	21

Table 1

STUDENT_COURSE

STUD_NO	COURSE_NO	COURSE_NAME
1	C1	DBMS
2	C2	Computer Networks
1	C2	Computer Networks

Table 2

Example: - EMPLOYEE_DETAIL

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Super key in the table above:

{EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP} ....so on.

Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non prime.

Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID.

The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on

super key (EMP_ID). It violates the rule of third normal form.
That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE Table: -

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

Super key in the table above:
{EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP} ....so on .
Candidate key: {EMP_ID}
Non-prime attributes: In the given table, all attributes except EMP_ID are non prime.
Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID.The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key (EMP_ID). It violates the rule of third normal form.
That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE Table: -

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP Table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

It is an advance version of 3NF that’s it is also referred as 3.5NF.

A table complies with BCNF if it is in 3NF and for every functional dependency X->Y, X should be the super key of the table.

Functional Dependency

Functional Dependency (FD) determines the relation of one attribute to another attribute in a database management system (DBMS) system.

Functional dependency helps you to maintain the quality of data in the database.

A functional dependency is denoted by an arrow →. The functional dependency of X on Y is represented by X → Y.

Example: Suppose there is a company wherein employees work in more than one department. They store the data like this:

emp_id	emp_nationality	emp_dept	dept_type	dept_no_of_emp
1001	Austrian	Production and planning	D001	200
1001	Austrian	stores	D001	250
1002	American	design and technical support	D134	100
1002	American	Purchasing department	D134	600

Functional dependencies in the table above:

emp_id -> emp_nationality

emp_dept -> {dept_type, dept_no_of_emp}.

Candidate key: {emp_id, emp_dept}

The table is not in BCNF as neither emp_id nor emp_dept alone are keys.

To make the table comply with BCNF we can break the table in three tables like this:

emp_nationality table:

emp_id	emp_nationality
1001	Austrian
1002	American

emp_dept table:

emp_dept	dept_type	dept_no_of_emp
Production and planning	D001	200
Stores	D001	250
design and technical support	D134	100
Purchasing department	D134	600

emp_dept_mapping table:

emp_id	emp_dept
1001	Production and planning
1001	Stores
1002	design and technical support
1002	Purchasing department

Functional dependencies:

emp_id -> emp_nationality

emp_dept -> {dept_type, dept_no_of_emp}

Candidate keys:

For first table: emp_id

For second table: emp_dept

For third table: {emp_id, emp_dept}

This is now in BCNF as in both the functional dependencies left side part is a key.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-3 (Schema Refinement & Normal Form)

Normalization(4NF)

LECTURE NO: -5

Fourth normal form (4NF)

• A relation will be in 4NF if it is in Boyce Codd normal form and has no multi valued dependency.
• For a dependency A → B, if for a single value of A, multiple values of B exist, then the relation will be a multi-valued dependency.

Multivalued Dependency
Multivalued dependency occurs when two attributes in a table are independent of each other but, both depend on a third attribute.
A multivalued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.

Example: Suppose there is a bike manufacturer company which produces two colors (white and black) of each model every year.

BIKE_MODEL	MANUF_YEAR	COLOR
M2011	2008	White
M2001	2008	Black
M3001	2013	White
M3001	2013	Black
M4006	2017	White
M4006	2017	Black

• Here columns COLOR and MANUF_YEAR are dependent on BIKE_MODEL and independent of each other.
• In this case, these two columns can be called as multivalued dependent on BIKE_MODEL. The representation of these dependencies is shown below:
• BIKE_MODEL → → MANUF_YEAR
• BIKE_MODEL → → COLOR

• This can be read as "BIKE_MODEL multidetermined MANUF_YEAR" and "BIKE_MODEL multidetermined COLOR".

Example: student

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

• The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.

• In the STUDENT relation, a student with STU_ID, 21 contains two courses, Computer and Math and two hobbies, Dancing and Singing. So there is a Multi-valued dependency on STU_ID, which leads to unnecessary repetition of data.

So to make the above table into 4NF, we can decompose it into two tables:

STUDENT_COURSE

STU_ID	COURSE
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

STUDENT_HOBBY

STU_ID	HOBBY
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.

2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-3 (Schema Refinement & Normal Form)

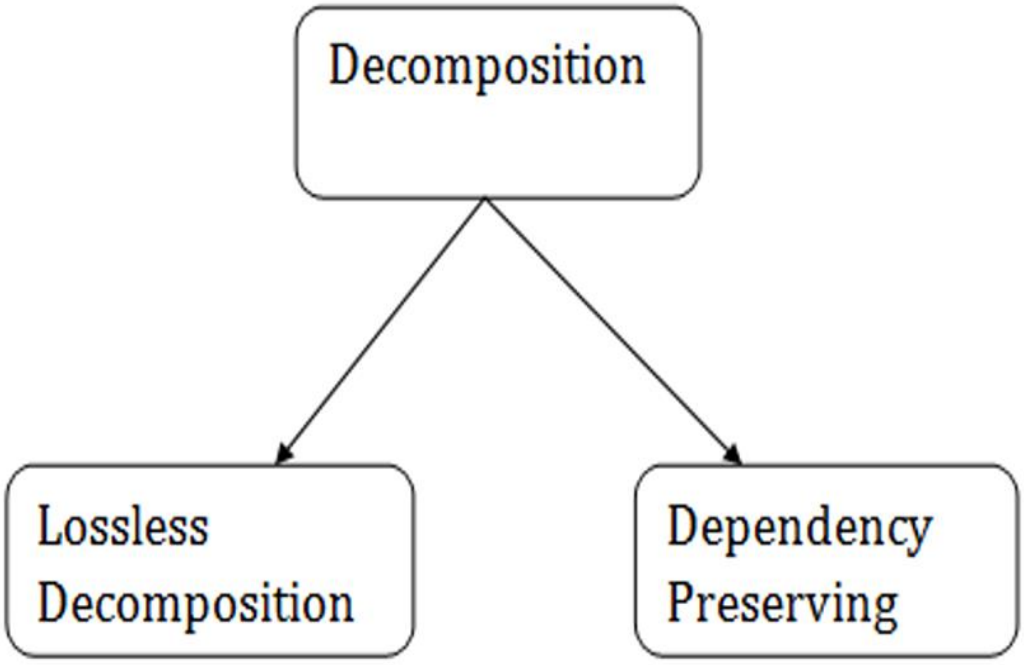
Decomposition into BCNF and 3-NF

LECTURE NO: -6

Relational Decomposition

When a relation in the relational model is not in appropriate normal form then the decomposition of a relation is required.
In a database, it breaks the table into multiple tables. If the relation has no proper decomposition, then it may lead to problems like loss of information. Decomposition is used to eliminate some of the problems of bad design like anomalies, inconsistencies, and redundancy.

Types of Decomposition



Lossless Decomposition

1. If the information is not lost from the relation that is decomposed, then the decomposition will be lossless.
2. The lossless decomposition guarantees that the join of relations will result in the same relation as it was decomposed.
3. The relation is said to be lossless decomposition if natural joins of all the decomposition give the original relation.

EMPLOYEE_DEPARTMENT table:

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY	DEPT_ID	DEPT_NAME
22	Denim	28	Mumbai	827	Sales
33	Alina	25	Delhi	438	Marketing
46	Stephan	30	Bangalore	869	Finance
52	Katherine	36	Mumbai	575	Production
60	Jack	40	Noida	678	Testing

The above relation is decomposed into two relations EMPLOYEE and DEPARTMENT

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY
22	Denim	28	Mumbai
33	Alina	25	Delhi
46	Stephan	30	Bangalore
52	Katherine	36	Mumbai
60	Jack	40	Noida

DEPARTMENT table

DEPT_ID	EMP_ID	DEPT_NAME
827	22	Sales
438	33	Marketing
869	46	Finance
575	52	Production
678	60	Testing

Now, when these two relations are joined on the common column "EMP_ID", then the resultant relation will look like:

Employee ⋈ Department

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY	DEPT_ID	DEPT_NAME
22	Denim	28	Mumbai	827	Sales
33	Alina	25	Delhi	438	Marketing
46	Stephan	30	Bangalore	869	Finance
52	Katherine	36	Mumbai	575	Production
60	Jack	40	Noida	678	Testing

Hence, the decomposition is Lossless join decomposition.

Dependency Preserving

1. It is an important constraint of the database.
2. In the dependency preservation, at least one decomposed table must satisfy every dependency.
3. If a relation R is decomposed into relation R1 and R2, then the dependencies of R either must be a part of R1 or R2 or must be derivable from the combination of functional dependencies of R1 and R2.
4. For example, suppose there is a relation R (A, B, C, D) with functional dependency set (A->BC). The relational R is decomposed into R1(ABC) and R2(AD) which is dependency preserving because FD A->BC is a part of relation

R1(ABC).

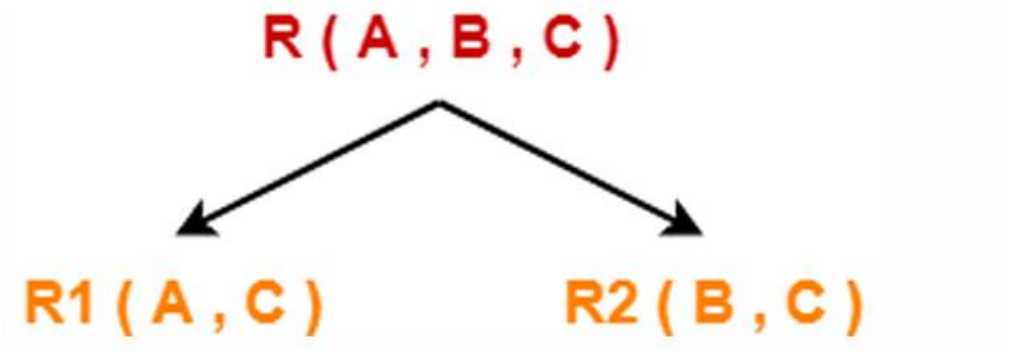
A B C

1 2 1

2 5 3

3 3 3

Consider this relation is decomposed into two sub relations as R1(A, C) and R2(B, C)-



The two sub relations are-

R1(A , C) **R2(B , C)**

A C **B C**

2 1	1 1
5 3	2 3
3 3	3 3

1.Now, let us check whether this decomposition is lossy or not.
2.For lossy decomposition, we must have-
R1 ⋈ R2 ⊃ R
Now, if we perform the natural join (⋈) of the sub relations R1 and R2 we get-

A B C
1 2 1
2 5 3
2 3 3
3 5 3
3 3 3

References: -
1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).